

Sistemas Operativos

Departamento de Computación - FCEyN - UBA
Segundo cuatrimestre de 2025

Nombre y apellido:

Nº orden

L.U.

Primer recuperatorio - 2do cuatrimestre de 2025

- Cada ejercicio debe realizarse en hojas separadas y numeradas. Debe identificarse cada hoja con nombre, apellido, LU y número de orden. Hojas sin identificar no serán corregidas. Completar en la hoja de enunciado la cantidad total de hojas entregadas y entregarla junto al examen, la misma no se incluye en la cantidad total de hojas entregadas.
- La devolución de los exámenes corregidos es personal. Los pedidos de revisión se realizarán por escrito, sin retirar el examen corregido del aula.
- Cada ejercicio se califica con Bien, Regular o Mal. La división de los ejercicios en incisos es *meramente* orientativa y cada ejercicio se califica globalmente. El parcial se aprueba con un mínimo de 2 ejercicios Bien y 1 Regular.

Ejercicio 1.

Se desea modelar el funcionamiento de una planta farmacéutica. Esta planta cuenta con un depósito para N lotes de insumos, otro depósito para M lotes de producto (el medicamento fabricado) y una línea de fabricación en la que se toma un lote de insumos (con `tomarLoteInsumos()`), se fabrican los compuestos, se hacen los comprimidos y se arman los blisters (todo con `fabricar()`), dando como resultado un lote de blisters terminados para ubicar en el depósito correspondiente (con `depositarLoteFabricado()`). Se puede fabricar un lote de medicamento sólo si i) el depósito de producto tiene espacio para el nuevo lote, y ii) hay un lote de insumos disponible en el depósito de insumos.

Se pide:

- Modelar mediante semáforos el funcionamiento de la planta, siguiendo la descripción dada.
- Modelar la llegada de un nuevo lote de insumos, que debe añadirlo al depósito de insumos si hay lugar. En caso de que el depósito de insumos se encuentre lleno, el lote recibido se pierde.
- Modelar la venta de un lote de producto, liberando espacio en el depósito de producto. Si no hay lotes en stock, el pedido de venta se pierde.

Ejercicio 2.

En una central atómica se desea controlar el estado del reactor nuclear. Hay un sistema de mediciones que escribe periódicamente los valores medidos por `stdout`. Para analizar y controlar dicha salida se tienen dos funciones:

```
float costosa(float value);  
float aproximada(float value);
```

- Se desea implementar un programa que utilice dos subprocesos hijos para computar concurrentemente de ambas maneras (un hijo debe aplicar la función `costosa()`, y el otro la función `aproximada()`, sobre cada medición). El padre, a su vez, debe estar constantemente obteniendo valores de `stdin`. Puede utilizar `scanf("%f", &x)` para leer un dato de `stdin` y convertirlo a un `float`. Cada hijo es responsable de escribir a `stdout` los resultados de sus propios cálculos.
- Modificar el programa anterior utilizando las señales `SIGUSR1` y `SIGUSR2` de modo tal que cada subproceso notifique al proceso padre tan pronto como tenga un resultado, y sea el padre quien muestre todos los resultados por `stdout`.

Sugerencia: numerar las líneas de código para referenciarlas adecuadamente al explicar la solución y/o al proponer modificaciones.

Ejercicio 3.

Responder verdadero o falso justificando:

- El programa `int main(int argc, char *argv[]) { printf("Hola"); return 0; }` no hace ninguna `syscall`.

- b) Si un programa ejecuta `fork()` las variables definidas antes son compartidas mientras que las modificadas luego del `fork()` no se comparten.
- c) Cualquier implementación de `fork()` debe crear otro proceso igual al que lo ejecuta y por ende se duplica la memoria consumida en el sistema.
- d) Si un proceso le pasa a otro a través de un pipe el puntero a una dirección de memoria, el segundo proceso puede leer de esa dirección el valor que haya puesto el proceso padre.
- e) Si un proceso consigue un file descriptor y luego ejecuta `fork()`, puede usar ese file descriptor para comunicar a los procesos padre e hijo.
- f) Un sistema operativo implementa la syscall `multifork()`, que lanza dos hijos en una sola llamada. El mecanismo de implementación está basado en `fork()`. Por ende, ambos hijos comparten las variables.
- g) Una *syscall* toma más tiempo de ejecución que una simple llamada a función.